



Python Course 2025/26/2

Lecture 6: OOP, Packages, and Project Structure

Course coordinator: Dr Mate Tejfel

Lecturer: Zoltan Kegli



Lecture Overview

Agenda

- OOP foundations: classes, objects, state, inheritance
- Modules, packages, and project structure
- Dependency management with `pip`, `requirements.txt`, virtual environments, `uv`, and `pixi`
- Type hints, PEP 8, and docstrings
- Command-line apps and the to-do package architecture

Learning objective

Design small object models, split them into packages, isolate dependencies safely, and wire them into a maintainable command-line application.

By the end, you can

Model simple domains with classes, organize code into modules and packages, manage dependencies with modern Python tools, and structure a small CLI project for clean growth.



OOP Foundations

From functions and data to cooperating objects

Python Is Multi-Paradigm

Python supports many styles

- Imperative: step-by-step instructions
- Functional: functions as values, mapping, filtering
- Declarative: comprehensions and generators
- Event-driven: common in GUIs and web frameworks
- Object-oriented: data and behavior live together

Why focus on OOP today?

Small scripts can stay procedural, but bigger programs need a cleaner way to model state, behavior, and collaboration.

Today's mental shift

Instead of asking only "Which function runs next?", we also ask "Which object owns this data, and who should change it?"

Why OOP When Programs Grow?

Procedural code is great when

- The task is small and linear
- State is simple and short-lived
- Only a few functions need to cooperate

But bigger projects hurt

Games, web apps, and data tools accumulate state. When data and behavior are scattered across many functions, code becomes harder to extend and debug.

Procedural Programming (Recipe)



- Focuses on steps/sequences
- Data and functions separate
- Example: "Step 1: Mix flour & sugar. Step 2: Bake at 350°F."

Object-Oriented Programming (Kitchen Station)



- Focuses on objects/classes
- Data and methods encapsulated
- Example: "BakeryStation" class with mix(), bake(), plate() methods.



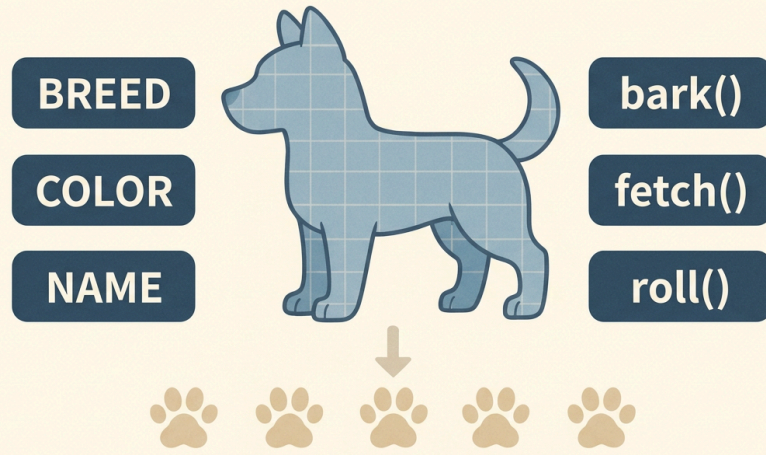
Key Differences

- Procedural: Linear flow; OOP: Modular reuse

Class vs Object

Class = blueprint

Dog Class (Template)



A class describes what data and behavior instances should have.

Object = one concrete instance

Dog Object (Instance)



An object stores actual values like a specific name, age, or breed.

Common confusion: a class is not "one special object". It is the reusable definition used to create many objects.

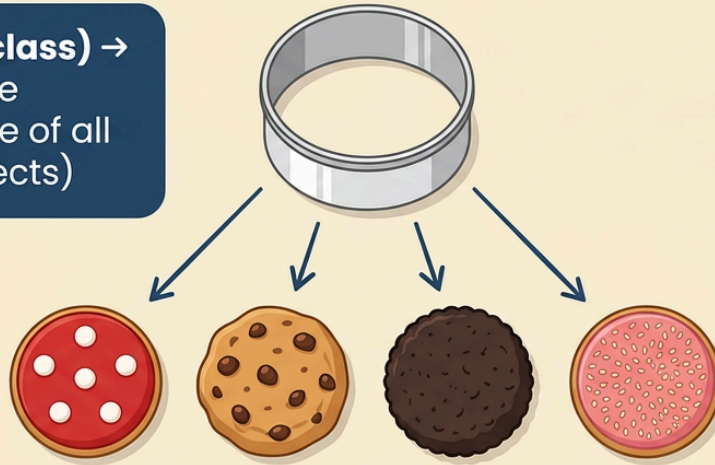
Fun Python fact: classes are objects too, so you can store a class in a variable, pass it to a function, or inspect it with `type()`.

Analogy: Cookie Cutter and Cookies



What Are Classes & Objects In Python?

Cookie cutter (class) →
Defines the
blueprint/shape of all
cookies (objects)



Different cookies (objects) → Have same blueprint but vary in flavor, toppings,

The cutter defines the shared shape; each cookie is a separate object with its own toppings, color, or flavor.

Anatomy of a Python Class

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print("Woof!")

    def birthday(self):
        self.age += 1
```

Methods are functions inside the class. Their first parameter refers to the current object.

Four core pieces

- `class Dog:` creates a new type
- `__init__` runs when an object is created
- `self.name`, `self.age` are attributes
- `bark()`, `birthday()` are behaviors

Misconception to avoid: `self` is not a keyword like `if`. It is the reference to the current instance by convention.



Objects Remember Their Own State

```
mochi = Dog("Mochi", 2)
print(mochi.status())

mochi.walk(2)
mochi.feed(150)
mochi.learn("roll over")

print(mochi.status())
```

Calls on the same object update the same stored attributes.

Encapsulation

- Data lives inside the object
- Methods are the approved ways to change that data
- The object's state evolves over time

OOP feels natural when state and behavior belong together: the dog walks, the dog gets hungry, the dog learns tricks.



Objects Can Collaborate

```
adopted = shelter.adopt(alice, dogs[0])  
print(adopted.name, "→", alice.name)  
print([d.name for d in shelter.dogs])
```

Dog

Stores data about one dog.

Shelter

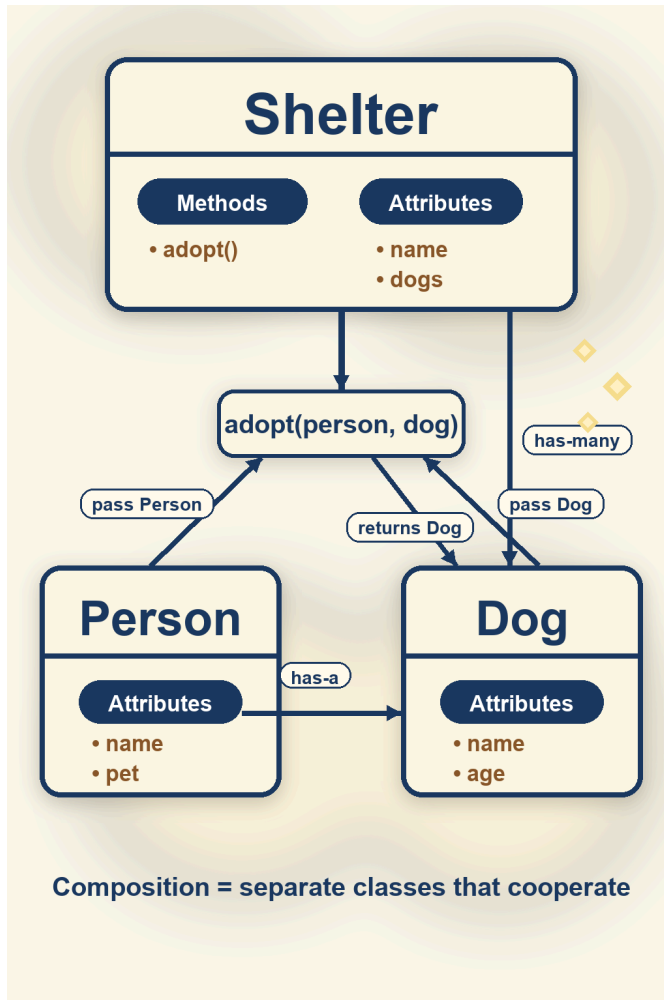
Owns a collection of dogs and manages adoption.

Person

May contain a reference to a pet.

Objects can be passed into methods, stored inside other objects, and returned from method calls. That is how small models become systems.

Composition: A Relationship Map



Four collaboration patterns

- **has-a** : one object keeps a reference to another
- **has-many** : one object manages a collection of others
- **Method parameter**: a method receives other objects so it can coordinate them
- **Method return**: a call can hand back the object it just worked with

Composition keeps classes independent. A shelter is not a dog and a person is not a dog, but they can still cooperate by storing references and calling methods.

Inheritance: Reuse With Care

When it helps

- A child class is a more specific version of the parent
- The child should reuse behavior and data layout
- The relationship is truly `is-a`

Good example: `Dog` is an `Animal`. Bad example: a shelter is not an animal.

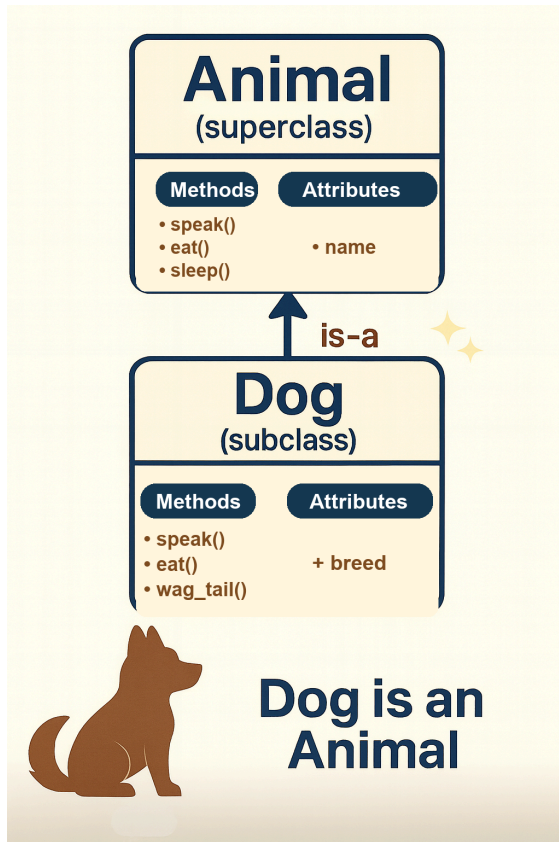
```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed

    def speak(self):
        print("Woof!")
```

`super()` lets the child reuse the parent's setup logic.

What Changes in the Child Class?



Four common cases

- Inherited untouched: the child keeps a parent behavior as-is
- Overridden: the child replaces a parent behavior with a more specific version
- Extended: the child starts with the parent behavior, then adds extra work
- New child-only method: the child adds behavior the parent never had

Inheritance is about specialization. If you only want one object to contain another, composition is often the better tool.

Checkpoint: OOP Foundations



Q1

What is printed?

```
ozzy = Dog("Ozzy", 2)
ozzy.birthday()
print(ozzy.age)
```

- A) 2
- B) 3
- C) "Happy Birthday"
- D) Dog years only

Show Answer

Q2

If `Dog` inherits from `Animal` and only overrides `speak()`, what happens when `my_dog.sleep()` runs?

- A) It uses the inherited `sleep()` method
- B) Python raises `MissingDogNapError`
- C) The child class forgets all parent methods
- D) It only works on Tuesdays

Show Answer



Checkpoint: OOP Foundations II

Q3

Which statement is the best description of a class?

- A) One specific dog named Mochi
- B) A blueprint used to create objects with shared structure and behavior
- C) A list that happens to bark
- D) A cookie with admin rights

Show Answer

Q4

In the shelter example, what kind of relationship is shown when `person.pet = dog` stores a dog inside a person object?

- A) Inheritance
- B) Composition / has-a relationship
- C) SyntaxError
- D) Bark-driven development

Show Answer



OOP: Summary

Model

- Class = blueprint
- Object = one instance
- Attributes store state

Behavior

- Methods act on the current object
- `self` refers to that object
- State changes stay bundled with data

Relationships

- Objects can collaborate
- Composition models "has-a"
- Inheritance models "is-a"

When you identify the right nouns first, the verbs often become obvious. That is the practical value of OOP.



Modules, Packages, and Environments

How Python projects stay organized and reproducible

A File Is a Module

Module basics

- Any `.py` file can be imported as a module
- Import executes the module once, then exposes its names
- The module name becomes a namespace
- Namespaces reduce accidental name collisions

```
# my_math.py
PI = 3.14159

def area_of_circle(radius):
    return PI * (radius ** 2)

# main.py
import my_math
print(my_math.area_of_circle(5))
```

The prefix tells readers exactly where a function or constant came from.

Fun Python fact: imported modules are cached in `sys.modules`, so importing the same module again usually reuses it instead of rerunning all its top-level code.



Three Import Patterns

Pattern	Example	Best when	Trade-off
<code>import module</code>	<code>import math</code>	Clarity matters most	More typing at call sites
<code>from module import name</code>	<code>from math import sqrt</code>	You want short local names	Origin becomes less visible
<code>import module as alias</code>	<code>import numpy as np</code>	The module name is long or widely aliased	Still need to learn the alias convention

Default to explicit imports first. Reach for shorter styles when the codebase has a clear convention.



A Folder Can Be a Package

```
todo-app-example/  
├── main.py  
└── todo/  
    ├── __init__.py  
    ├── cli.py  
    ├── manager.py  
    └── model.py
```

Packages help us move from "one big script" to clearly separated responsibilities.

Why package code?

- Related modules live together
- Imports become predictable
- Large apps stay navigable
- `__init__.py` marks package boundaries in beginner-friendly examples

`from todo.cli import main` is clearer than burying CLI, storage, and models in one file.

The Entry Point Guard

```
from todo.cli import main

if __name__ == "__main__":
    main()
```

The file stays tiny: it imports the real CLI entry function, then decides whether to run it.

What those two lines do

- When you run this file directly, `__name__` becomes `"__main__"`
- The condition becomes true, so `main()` starts the app
- If another module imports this file, the condition is false
- That prevents accidental side effects during import

Imports should usually expose names, not unexpectedly run the whole program. The `__main__` guard keeps startup logic explicit.



Three Sources of Python Code

Standard library

`json`, `sys`, `pathlib`

Ships with Python itself.

Third-party packages

`requests`, `numpy`, `rich`

Installed separately from package indexes.

Local application code

`todo.model`, `todo.manager`

Written by your team inside the project.

Important distinction: a Python package in your source tree and a downloadable package on PyPI are related ideas, but they are not the same thing.



Classic Workflow: pip and requirements.txt

```
pip install requests
pip install rich

pip freeze > requirements.txt
pip install -r requirements.txt
```

pip installs packages. requirements.txt records which versions you want to recreate.

What this gives you

- Repeatable installs on another machine
- A shareable dependency list
- A baseline for collaboration and deployment

Limit: requirements.txt does not isolate dependencies by itself. It should be paired with a virtual environment.



Virtual Environments Prevent Dependency Collisions

Why they exist

- Project A might need one package version
- Project B might need another
- Global installs mix everything together
- A virtual environment gives one project its own Python world

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# later
deactivate
```

Think of a virtual environment as a per-project backpack: interpreter, installed packages, and scripts stay local to that project.

Fun environment fact: a virtual environment is mostly just a folder on disk, which is why removing `.venv` cleanly removes a project's local packages without touching other projects.



Modern Workflow: uv and pixi

⚡ uv

- Great default for Python-first projects
- Creates environments quickly
- Handles install, sync, and run workflows

```
uv venv
uv add requests
uv run python app.py
uv sync
```

📦 pixi

- Useful when conda-level or compiled system dependencies matter
- Works well for scientific or mixed-language stacks
- Can manage both conda and PyPI packages

```
pixi init
pixi add numpy
pixi add --pypi rich
pixi run python app.py
```

Course recommendation: prefer `uv` by default, and reach for `pixi` only when you need non-PyPI or system-level dependencies.



Checkpoint: Modules and Environments

Q5

Which import keeps the origin visible at every use site?

- A) `import math` then `math.sqrt(16)`
- B) `from math import sqrt` then `sqrt(16)`
- C) `from math import *`
- D) Whisper the function name and hope Python hears you

Show Answer

Q6

What problem does a virtual environment primarily solve?

- A) It makes Python run in virtual reality
- B) It isolates project-specific packages from other projects
- C) It replaces `requirements.txt` as documentation
- D) It automatically writes all your docstrings

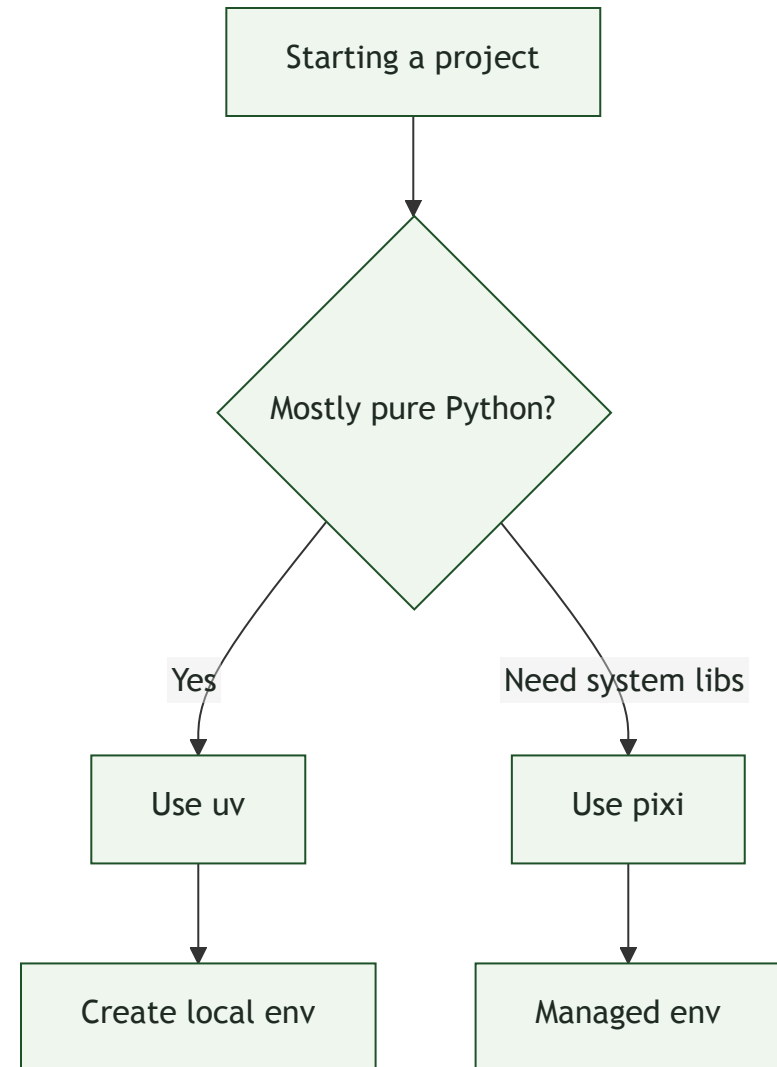
Show Answer



Packages and Environments: Summary

Keep these layers separate

- Module: one Python file
- Package: a folder of related modules
- Installer + environment solve different problems





Type Hints, Style, and Docs

Making code easier for humans and tools to understand

Why Add Type Hints?

```
def add(a, b):  
    return a + b  
  
add(5, 10)  
add("hello", "world")  
add(5, "world")
```

All three calls are syntactically allowed, but not all three make semantic sense.

Hints answer three questions

- What kind of data does this function expect?
- What kind of result should it return?
- What mistakes can tools catch before runtime?

Type hints are a communication tool for humans, editors, and static checkers.

Modern Hint Syntax

```
def process_names(names: list[str]) → None:
    for name in names:
        print(name.upper())

def print_prices(prices: dict[str, float]) → None:
    for item, price in prices.items():
        print(f"{item}: ${price:.2f}")

def greeting(name: str) → str:
    return "Hello, " + name
```

What to notice

- Parameter hint: `name: str`
- Return hint: `→ str`
- Containers can also be typed: `list[str]`

You will still meet older container hints like `List[str]` and `Dict[str, float]` in tutorials and existing code. Modern Python often prefers built-in generics like `list[str]` and `dict[str, float]`, so it is worth recognizing both styles.



Hints Help Tools, Not the Interpreter

```
def add(a: int, b: int) → int:  
    return a + b  
  
result = add(5, "world")
```

Python accepts the function definition. The failure comes later when incompatible values are actually combined.

Where hints shine

- Editor autocomplete and hover help
- Static checking with tools like mypy or pyright
- Clearer APIs for teammates and future you

Misconception: type hints do not make Python "strictly typed". They make code easier to reason about before bugs reach runtime.

Fun Python fact: type hints are available at runtime through `__annotations__`, which is why editors, checkers, and some frameworks can inspect them.



PEP 8 Makes Code Easier to Scan

High-value rules

- 4 spaces for indentation
- Spaces around operators and after commas
- `lowercase_with_underscores` for functions and variables
- `PascalCase` for classes
- Imports at the top, grouped by origin

```
# harder to scan
def CalculateAverage(NUMBERS):
    Total=0
    for n in NUMBERS: Total+=n
    return Total/len(NUMBERS)

# clearer
def calculate_average(numbers):
    total = 0
    for number in numbers:
        total += number
    return total / len(numbers)
```

Docstrings Explain What and Why

```
def calculate_average(numbers: list[float]) → float:
    """Calculate the average of a list of numbers.

    Args:
        numbers: Input values.

    Returns:
        The arithmetic mean.
    """
    return sum(numbers) / len(numbers)
```

Comments vs docstrings

- Comments help developers follow tricky implementation details
- Docstrings describe the public contract of a module, class, or function
- Docstrings are available through `help()` and `__doc__`

A good docstring starts with a one-line summary, then expands only if readers need more context.



Checkpoint: Type Hints, Style, and Docs

Q7

Which statement about type hints is correct?

- A) They are always enforced by the Python interpreter
- B) They help static tools and readers understand intent
- C) They replace unit tests completely
- D) They work only after sacrificing a rubber duck

Show Answer

Q8

Where should a function docstring go?

- A) As the first string inside the function body
- B) At the end of the file, next to the imports
- C) Only in a separate README
- D) Hidden inside a stack trace for dramatic effect

Show Answer



Command-Line Apps

Turning Python scripts into small developer tools

Reading sys.argv

```
import sys

print(sys.argv)

# python calc.py 4 + 5
# ['calc.py', '4', '+', '5']
```

Key facts

- `sys.argv[0]` is the script name/path
- All later items are command-line arguments
- Every argument arrives as a string
- You still need validation and conversion

`sys.argv` is low-level but important. It shows what command-line tools really receive from the shell.

Fun shell fact: quotes are handled before Python starts, so `"buy milk"` can arrive as one argument instead of two.

Real Example: calc.py

```
if len(sys.argv) != 4:
    print("Usage: python3 calc.py <num1> <op> <num2>")
    sys.exit(1)

try:
    num1 = float(sys.argv[1])
    num2 = float(sys.argv[3])
except ValueError:
    print("Error: Both numbers must be numeric.")
    sys.exit(1)
```

What it already handles

- Wrong number of arguments
- Non-numeric input
- Unknown operators

Good review question: what still fails? Division by zero is still possible when the operator is `/`.



When argparse Beats Manual Parsing

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("num1", type=float)
parser.add_argument("op", choices=["+", "-", "*", "/"])
parser.add_argument("num2", type=float)
args = parser.parse_args()
```

Why upgrade?

- Automatic `--help` output
- Built-in type conversion
- Validation rules like `choices=`
- Cleaner code once commands and flags multiply

Learn `sys.argv` first so you understand the raw input. Then use `argparse` when real tools need safer parsing.



Checkpoint: Command-Line Apps

Q9

What does `sys.argv[0]` usually contain?

- A) The first user-supplied argument
- B) The script name or path
- C) The operating system version
- D) The shell's emotional state

Show Answer

Q10

Which bug can still happen in the provided `calc.py` after numeric conversion succeeds?

- A) ZeroDivisionError when dividing by zero
- B) Python forgets what numbers are
- C) The plus sign becomes a string only on leap years
- D) The shell silently rewrites the source file

Show Answer



To-Do App Integration

Putting OOP, packaging, CLI design, and persistence together



Model the Problem With a Task Class

```
class Task:
    def __init__(self, description):
        self.description = description
        self.completed = False

    def complete(self):
        self.completed = True
```

Why start here?

- A task is the core noun in the app
- The object owns its own state
- Behavior like `complete()` belongs next to that state

This is OOP paying off: the app stops being "a pile of dicts" and starts having explicit domain objects.



Persistence Lives in the Manager

```
class TodoManager:
    def add_task(self, description):
        task = Task(description)
        self.tasks.append(task)
        self.save_to_file()

    def load_from_file(self):
        if os.path.exists(self.file_path):
            data = json.load(f)
            self.tasks = [Task.from_dict(item) for item in data]
```

Manager responsibilities

- Own the task collection
- Save and load JSON
- Validate task indices
- Return clear success/error messages

Notice how persistence and validation are centralized. CLI code stays simpler because storage logic is not copied into every command branch.

Fun data fact: JSON knows strings, numbers, booleans, lists, and dictionaries, but not your custom Python classes directly, so apps usually convert objects before saving.



The CLI Is Just the Front Door

```
command = argv[1].lower()

if command == "add":
    self.manager.add_task(description)
elif command == "list":
    tasks = self.manager.list_tasks()
elif command == "complete":
    success, message = self.manager.complete_task(task_index)
```

Separation of concerns

- CLI parses arguments and prints messages
- Manager owns business logic
- Model represents one task

Packages make this architecture easy to read: each module has a job, and imports show who depends on whom.



Package Structure Makes the App Scalable

```
src/todo-app-example/  
├── main.py  
└── todo/  
    ├── __init__.py  
    ├── cli.py  
    ├── manager.py  
    └── model.py
```

Import flow

```
main.py imports todo.cli.  
todo.cli imports TodoManager.  
todo.manager imports Task.
```

That structure mirrors the architecture in your head, which is exactly what good packaging should do.



Checkpoint: To-Do App Integration

Q11

Which class should be responsible for saving tasks to disk?

- A) `Task` , because every task deserves a hard drive
- B) `TodoManager` , because it manages the collection and persistence
- C) `TodoCLI` , because printing text and file I/O are the same thing
- D) `__init__.py` , because it sounds official

Show Answer

Q12

Why is it useful that `main.py` only imports and calls `todo.cli.main()`?

- A) It keeps the entry point tiny and pushes details into the package
- B) It makes Python execute twice as fast instantly
- C) It prevents all future bugs forever
- D) It unlocks admin mode

Show Answer



Lecture 6: Big Picture

Model

- Use classes to represent important nouns
- Let methods own behavior that changes state

Organize

- Split code into modules and packages
- Use isolated environments for reproducible dependencies

Communicate

- Type hints clarify intent
- PEP 8 and docstrings improve maintainability

The to-do app is the bridge between all these ideas: OOP gives structure, packages give shape, and CLI plus persistence make the program actually useful.



Thank You for Your Attention!

"The real question is not whether machines think but whether men do."

-- B. F. Skinner

oop

packages

type hints

cli apps